

What I actually did in SGLang

Plain English. What SGLang is, which part of it I touched, what I found, what I got wrong, and honestly whether it was any good. Written 2026-09-10, after the work was finished.

1 • What SGLang is

SGLang is an **inference server** for large language models. It is the software a company runs so that many users can send prompts to one model at the same time and get answers back fast. vLLM is its main competitor. It is a big, serious project, backed by xAI, Oracle, LinkedIn and Cursor.

Think of it as the difference between a home kitchen and a restaurant kitchen. Running a model in plain PyTorch is the home kitchen: one dish at a time, no rush. SGLang is the restaurant: hundreds of orders at once, and every second matters.

2 • Which part of SGLang I worked on

I did **not** touch the model code, the maths kernels, or the core scheduler. I worked on the **startup configuration layer**: the code that runs once when the server boots and decides how to size things.

Two files specifically:

File	What it does
<code>arg_groups/memory_hook.py</code>	Decides how large a batch of requests the server will pre-optimize for
<code>model_executor/runner/base_cuda_graph_runner.py</code>	Builds the actual list of batch sizes, and cuts it down if the request pool is smaller

One sentence for an interview. "I worked on how SGLang chooses its decode CUDA-graph coverage at startup, and showed that the choice ignores model size."

3 • The idea you need: pre-recorded checklists

Every time the GPU generates one token, the CPU has to issue hundreds of small commands to it. Issuing those commands takes time, and the GPU sits idle waiting for them.

A **CUDA graph** is a fix for this. The server records the whole sequence of commands once, like writing down a checklist, and then replays the checklist instead of re-issuing every command. Much faster.

But there is a catch. A checklist only works for the exact batch size it was recorded for. A checklist for 8 requests does not work for 9. So at startup SGLang records a set of checklists, for example for batch sizes 1, 2, 4, 8, 12, 16, 24, 32, 40, 48.

The failure. If 60 requests arrive at once and the largest checklist is 48, there is no checklist to use. The server falls back to issuing every command by hand, one at a time. That is far slower. This is what people call falling off the cliff.

4 • The actual bug

Here is the question: how does SGLang decide how many checklists to prepare?

It looks at how much memory the GPU has. That is all. A table in the code says: under 20 GB, prepare up to 8. Under 35 GB, prepare up to 48. Under 60 GB, prepare up to 32. And so on.

The size of the model never enters the decision. That is the bug.

Why it matters: a small model leaves a lot of spare memory, and spare memory is what lets the server run many requests at once. So a small model on a big card can easily reach a batch of 100 or more, while its

checklists stop at 48. A huge model on the same card can barely reach 10. Both get exactly the same checklists.

ANALOGY

A restaurant decides how many pre-written recipe cards to prepare based only on the size of its kitchen, never on how quick the dish is. A kitchen serving instant noodles can handle 100 orders at once but only has cards for 48. Past 48, the chef improvises every order from scratch and everything slows to a crawl.

5 · What I did, step by step

1. Found a real open bug report about this, filed by someone else on an L40 GPU (issue #33483).
2. Read the SGLang source myself and confirmed the mechanism, rather than trusting the report.
3. Noticed something the report did not cover: **four days before my run**, a maintainer had raised the setting for my memory tier from 24 to 48. But they justified it using only large models (9 billion parameters and up). Nobody had checked a small model.
4. Rented an RTX 4090 GPU on Vast.ai for about three hours.
5. Ran a tiny model (0.5 billion parameters) two ways on the same card in the same session: once with the default 48, once forced to 128.
6. Swept the request rate from 16 to 72 requests per second, two repeats at each point.
7. Wrote the results up honestly, posted them publicly, and published the code, the raw data and the server logs so anyone can repeat it.

Why same card, same session matters. An earlier experiment in this repo found two supposedly identical GPUs differing by 1.22x in speed. If I had measured the two settings on two different rented machines, the comparison would have been meaningless.

6 · What I found

The bug is real, and it survives the recent fix.

Requests per second	Default (48)	Widened (128)	How much worse
16	2.64 ms per token	2.63 ms	no difference
32	30.57 ms per token	5.31 ms	5.76x slower
48	40.99 ms	37.81 ms	almost none
72	58.70 ms	59.41 ms	none

At 32 requests per second the default setting was nearly **six times slower** per token. Same GPU, same model, same minute. The only difference was one configuration number.

7 · The two things I got wrong, and caught

This is the part I would actually point at, more than the 5.76x.

Mistake 1: I said widening was free. It is not.

I compared total GPU memory used before and after, saw almost no difference, and concluded there was no cost. That was the wrong thing to measure. SGLang automatically shrinks its memory pool by roughly the amount the checklists grow, so the **total** can never change much. I was measuring the one number the system deliberately holds still.

The real cost was **14,110 tokens of cache capacity**, about 1.1% of the pool, plus half a second of startup time. Once I measured the right variable, it matched SGLang's own internal prediction almost exactly.

Mistake 2: I thought the problem disappeared at high load.

The gap opened at 32 requests per second and closed again by 48, which looked like the effect had limits. It does not. Past 48 the GPU is simply **saturated**, maxed out at about 9,700 tokens per second either way, so everything is stuck waiting in a queue and the checklists cannot help.

So the bug bites **below** saturation, which is exactly the load a real server runs at. That makes the finding more important, not less.

Both corrections were posted publicly on the same issue thread, with the data behind them, rather than quietly fixed.

8 · What I did NOT do

I did not get code merged into SGLang. There is no pull request of mine in the project. This should be said plainly and not dressed up.

The plan was to submit a small fix that warns the operator when this happens. While preparing it, I found that **that exact fix already existed** as pull request #33900, written a month earlier by the person who filed the original bug report, and sitting completely unreviewed. Writing a competing one would have been wasted effort.

So I looked at why it was unreviewed, and measured the project itself:

Measurement, 2026-09-08	Value
Open pull requests in SGLang	4,318
Of those, older than one month	2,446
First-time authors in a sample of 12 recent merges	0
Unassigned issues I checked that already had a pull request	5 of 7

The conclusion is uncomfortable but useful: **writing a good fix is not the hard part. Getting anyone to look at it is.** The outside contributions that do get merged are mostly from engineers at AMD, Intel and NPU vendors working on their own hardware, who already have relationships with the maintainers.

9 · So was it good work, honestly?

Yes, with one clear limit.

What is genuinely good:

- I read a real production system's source and understood a mechanism, rather than reading a blog post about it.
- I found an angle nobody else had: the setting had just been changed using only large models, and I tested the case its author did not.
- The experiment design is sound. Same card, same session, repeats, both configurations swept across the same rates.
- I caught two of my own errors and corrected them in public with data. Most people quietly fix mistakes or never notice them.
- Everything is reproducible: code, raw CSV, server logs, and a runbook, all published.
- The whole thing cost under \$2.

The limit: **this is a measurement, not a merged contribution.** If someone asks "have you contributed to open source," the honest answer today is "I published an independent reproduction of a bug in SGLang and it is on the public issue thread," not "I have code in SGLang."

For MS applications this is arguably the better artifact anyway. An admissions committee can read a blog post and a repository. They cannot read a pull request queue. What they are looking for is evidence that you can form a question, design a fair measurement, and report the result honestly including when it contradicts you. That is exactly what this shows.

10 · Where everything lives

Thing	Where
The write-up	blog/sglang-cuda-graph-cliff.md
The measurement code	scripts/sglang_cudagraph_cliff.py
Raw results	results/sglang_cliff.csv
Server logs	logs/sglang_cliff/
How to repeat it	docs/sglang-contribution-runbook.md
What was said upstream	docs/issue-33483-correspondence.md
The public thread	github.com/sgl-project/sglang/issues/33483

One open item: if no SGLang maintainer replies to the thread on or after 2026-09-15, ping it once, politely. Nobody from the team has replied since the issue was filed on 2026-08-04, across four comments, so expect silence rather than reading it as a rejection.